Most getters and constructors omitted for readability

Class — Boundary Classes

Class — Control Classes

Class — Entity Classes

<<boundary>>
Main
+main(String[] args): void

<<boundary>>
ConsoleUI
-BufferedReader input
-ConsoleView view
-boolean echo
+play(GameEngine game): void
-readWaiter(GameEngine game): void
-readChef(GameEngine game): void
-parseTarget(String[] parts, boolean wait): int
-readCommand(String role): String[]

<<boundary>>
ConsoleView
-int WIDTH
-PrintWriter output
+showWelcome(GameEngine game): void
+showTurnStart(int turn): void
+showState(GameEngine game): void
+showWaiter(GameState state): void
+showChef(GameState state): void
+showTurnEnd(GameEngine game, List<String> events): void
+showGoodbye(GameEngine game): void
+feedback(String message): void
+error(String message, boolean beforeTurn): void
+prompt(String role): void
+echo(String command): void

<<enumeration>>
WaiterTask
SEAT
ORDER
SERVE
WAIT

<<control>>
GameEngine
-GameSetup setup
-GameState state
-Waiter waiter
-Chef chef
-Cashier cashier
-int turn
-boolean turnOpen
-boolean waiterAssigned
-boolean chefAssigned
+startTurn(): void
+assignWaiter(WaiterTask task, int customerId): String
+assignChef(int orderId): String
+finishTurn(): List<String>
+isFinished(): boolean
+getResult(): GameResult
-requireOpen(): void

COMPOSITION: GameEngine creates and owns GameState, Waiter, Chef and Cashier

<<control>>
Cashier
+collectOldestEligible(GameState state, int turn): Receipt

<<entity>>
GameResult
-boolean victory
-int paidCount
-int revenueCents
-String average
+isVictory(): boolean
+describe(): String

<<entity>>
Game State
-Menu menu
-List<Table> tables
-List<Customer> customers
-Deque<Customer> waiting
-List<Order> orders
-List<Receipt> receipts
-int servedCount
-int unhappyDepartures
+averageSatisfactionText(): String
+customer(int id): Customer
+order(int id): Order
+orderFor(int customerId): Order
-getRevenueCents(): int
+getPaidCount(): int
+arrive(Customer c): void
+seat(Customer c): void
+addOrder(Order o): void
+quoteBill(Customer c, MenuItem item): int
+recordServed(): void
+recordPayment(Customer c, Order o, int turn): void
+recordDeparture(Customer c): void
-releaseTable(Customer c): void

COMPOSITION: GameState creates and owns its two Tables and its Receipts

<<control>>
Waiter
+seat(GameState state, Customer c): String
+takeOrder(GameState state, Customer c): String
+serve(GameState state, Customer c, int turn): String

<<entity>>
Receipt
-int customerId
-int cents
-int satisfaction
-int turn
+getCents(): int
+getSatisfaction(): int

<<control>>
Chef
+prepare(Order order) : String

ENCAPSULATION: status and completedUnits are private.
They change only through prepareOneUnit(), serve(), pay() and cancel(), which validate the transition.
Bill is locked at construction

<<entity>>
Table
-int id
-Customer customer
+isFree(): boolean
+seat(Customer c): void
+release(): void
+getCustomer(): Customer

<<entity>>
Order
-int id
-int customerId
-MenuItem item
-int billCents
-int completedUnits
-Status status
+canPrepare(): boolean
+prepareOneUnit(): void
+serve(): void
+pay(): void
+cancel(): void
-require(Status expected): void

<<control>>
GameSetup
+int MAX_TURNS
+getStageName(): String
+createMenu(): Menu
+createCustomer(int id): Customer
+hasArrival(int turn): boolean

<<entity>>
Dish
-String name
-int priceCents
-int preparationUnits
+getName(): String
+getPriceCents(): int
+getPreparationUnits(): int

<<entity>>
Menu
-List<MenuItem> items
+getItems(): List<MenuItem>
+get(String name) : MenuItem
+chooseFor(int customerId): MenuItem

<<abstract>>
Customer
-int id
-int satisfaction
-Status status
-int servedTurn
-int readyToPayTurn
+getType(): String
+waitingLoss(): int
+chooseMeal (Menu menu): MenuItem
+adjustBill(int cents): int
+isWaitingForService(): boolean
+seat(): void
+order(): void
+serve(int turn): void
+finishEating(int turn): boolean
+canPay(int turn): boolean
+pay(int turn): void
+waitAtTurnEnd(): boolean

<<enumeration>>
Order.Status
PLACED
PREPARING
READY
SERVED
PAID
CANCELLED

<<enumeration>>
Customer.Status
WAITING
SEATED
WAITING_FOR_FOOD
EATING
READY_TO_PAY
PAID
LEFT

<<Interface>>
MenuItem
+getName(): String
+getPreparationUnits(): int
+getPriceCents():int

<<entity>>
RegularCustomer
+getType(): String
+waitingLoss(): int

Most getters and constructors omitted for readability

Class — Boundary Classes

Class — Control Classes

Class — Entity Classes

<<boundary>>
Main
+main(String[] args): void

<<boundary>>
ConsoleUI
-BufferedReader input
-ConsoleView view
-boolean echo
+play(GameEngine game): void
-readWaiter(GameEngine game): void
-readChef(GameEngine game): void
-parseTarget(String[] parts, boolean wait): int
-readCommand(String role): String[]

<<boundary>>
ConsoleView
-int WIDTH
-PrintWriter output
+showWelcome(GameEngine game): void
+showTurnStart(int turn): void
+showState(GameEngine game): void
+showWaiter(GameState state): void
+showChef(GameState state): void
+showTurnEnd(GameEngine game, List<String> events): void
+showGoodbye(GameEngine game): void
+feedback(String message): void
+error(String message, boolean beforeTurn): void
+prompt(String role): void
+echo(String command): void

<<enumeration>>
WaiterTask
SEAT
ORDER
SERVE
WAIT

<<control>>
GameEngine
-GameSetup setup
-GameState state
-Waiter waiter
-Chef chef
-Cashier cashier
-int turn
-boolean turnOpen
-boolean waiterAssigned
-boolean chefAssigned
+startTurn(): void
+assignWaiter(WaiterTask task, int customerId): String
+assignChef(int orderId): String
+finishTurn(): List<String>
+isFinished(): boolean
+getResult(): GameResult
-requireOpen(): void

COMPOSITION: GameEngine creates and owns GameState, Waiter, Chef and Cashier

<<control>>
Cashier
+collectOldestEligible(GameState state, int turn): Receipt

<<entity>>
GameResult
-boolean victory
-int paidCount
-int revenueCents
-String average
+isVictory(): boolean
+describe(): String

<<entity>>
GameState
-Menu menu
-List<Table> tables
-List<Customer> customers
-Deque<Customer> waiting
-List<Order> orders
-List<Receipt> receipts
-int servedCount
-int unhappyDepartures
+averageSatisfactionText(): String
+customer(int id): Customer
+order(int id): Order
+orderFor(int customerId): Order
+getRevenueCents(): int
+getPaidCount(): int
+arrive(Customer c): void
+seat(Customer c): void
+addOrder(Order o): void
+quoteBill(Customer c, MenuItem item): int
+recordServed(): void
+recordPayment(Customer c, Order o, int turn): void
+recordDeparture(Customer c): void
-releaseTable(Customer c): void

COMPOSITION: GameState creates and owns its two Tables and its Receipts

<<control>>
Waiter
+seat(GameState state, Customer c): String
+takeOrder(GameState state, Customer c): String
+serve(GameState state, Customer c, int turn): String

<<entity>>
Receipt
-int customerId
-int cents
-int satisfaction
-int turn
+getCents(): int
+getSatisfaction(): int

<<control>>
Chef
+prepare(Order order) : String

ENCAPSULATION: status and completedUnits are private.
They change only through prepareOneUnit(), serve(), pay() and cancel(), which validate the transition.
Bill is locked at construction

<<entity>>
Table
-int id
-Customer customer
+isFree(): boolean
+seat(Customer c): void
+release(): void
+getCustomer(): Customer

<<entity>>
Order
-int id
-int customerId
-MenuItem item
-int billCents
-int completedUnits
-Status status
+canPrepare(): boolean
+prepareOneUnit(): void
+serve(): void
+pay(): void
+cancel(): void
-require(Status expected): void

<<control>>
GameSetup
+int MAX_TURNS
+getStageName(): String
+createMenu(): Menu
+createCustomer(int id): Customer
+hasArrival(int turn): boolean

<<entity>>
Dish
-String name
-int priceCents
-int preparationUnits
+getName(): String
+getPriceCents(): int
+getPreparationUnits(): int

<<entity>>
Menu
-List<MenuItem> items
+getItems(): List<MenuItem>
+get(String name) : MenuItem
+chooseFor(int customerId): MenuItem

<<enumeration>>
Order.Status
PLACED
PREPARING
READY
SERVED
PAID
CANCELLED

<<abstract>>
Customer
-int id
-int satisfaction
-Status status
-int servedTurn
-int readyToPayTurn
+getType(): String
+waitingLoss(): int
+chooseMeal (Menu menu): MenuItem
+adjustBill(int cents): int
+isWaitingForService(): boolean
+seat(): void
+order(): void
+serve(int turn): void
+finishEating(int turn): boolean
+canPay(int turn): boolean
+pay(int turn): void
+waitAtTurnEnd(): boolean

<<enumeration>>
Customer.Status
WAITING
SEATED
WAITING_FOR_FOOD
EATING
READY_TO_PAY
PAID
LEFT

<<Interface>>
MenuItem
+getName(): String
+getPreparationUnits(): int
+getPriceCents():int

<<entity>>
RegularCustomer
+getType(): String
+waitingLoss(): int

<<entity>>
VIPCustomer
+getType(): String
+waitingLoss(): int
+adjustBill(int cents): int

<<entity>>
CriticCustomer
+getType(): String
+waitingLoss(): int
+chooseMeal(Menu menu): MenuItem

POLYMORPHISM: Waiter calls chooseMeal(), GameState.quoteBill() calls adjustBill(), and Customer.waitAtTurnEnd() calls waitingLoss(), all on Customer.
VIP and Critic differ by override, with no type checks.

Most getters and constructors omitted for readability

Class — Boundary Classes

Class — Control Classes

Class — Entity Classes

<<boundary>>
Main
+main(String[] args): void

<<boundary>>
ConsoleUI
-BufferedReader input
-ConsoleView view
-boolean echo
+play(GameEngine game): void
-readTurnStart(GameEngine game): void
-readHost(GameEngine game): void
-readWaiter(GameEngine game): void
-readChef(GameEngine game): void
-parseTarget(String[] parts, boolean wait): int
-readCommand(String role): String[]

<<boundary>>
ConsoleView
-int WIDTH
-PrintWriter output
+showWelcome(GameEngine game): void
+showTurnStart(int turn): void
+showState(GameEngine game): void
+showHappyHourChoice(GameEngine game): void
+showHost(GameState state): void
+showWaiter(GameState state): void
+showChef(GameState state): void
+showTurnEnd(GameEngine game, List<String> events): void
+showGoodbye(GameEngine game): void
+feedback(String message): void
+error(String message, boolean beforeTurn): void
+prompt(String role): void
+echo(String command): void

<<enumeration>>
WaiterTask
SEAT
ORDER
SERVE
WAIT

<<control>>
GameEngine
-GameSetup setup
-GameState state
-HappyHourPolicy happyHour
-Host host
-Waiter waiter
-Chef chef
-Cashier cashier
-int turn
-boolean turnOpen
-boolean hostAssigned
-boolean waiterAssigned
-boolean chefAssigned
+startTurn(boolean activateHappyHour): void
+assignHost(int customerId): String
+assignWaiter(WaiterTask task, int customerId): String
+assignChef(int orderId): String
+finishTurn(): List<String>
+isFinished(): boolean
+getResult():GameResult
+getHappyHourRemaining(): int
+hasUsedHappyHour(): boolean
-requireOpen(): void

COMPOSITION: GameEngine creates and owns GameState, HappyHourPolicy and the four staff objects

<<control>>
Cashier
+collectOldestEligible(GameState state, int turn): Receipt

<<entity>>
GameResult
-boolean victory
-int paidCount
-int revenueCents
-String average
+isVictory(): boolean
+describe(): String

<<control>>
Host
+seat(GameState state, Customer c): String

<<entity>>
Game State
-Menu menu
-HappyHourPolicy pricing
-List<Table> tables
-List<Customer> customers
-Deque<Customer> waiting
-List<Order> orders
-List<Receipt> receipts
-int servedCount
-int unhappyDepartures
+averageSatisfactionText(): String
+customer(int id): Customer
+order(int id): Order
+orderFor(int customerId): Order
+getRevenueCents(): int
+getPaidCount(): int
+arrive(Customer c): void
+seat(Customer c): void
+addOrder(Order o): void
+quoteBill(Customer c, MenuItem i): int
+recordServed(): void
+recordPayment(Customer c, Order o, int turn): void
+recordDeparture(Customer c): void
-releaseTable(Customer c): void

COMPOSITION: GameState creates and owns its two Tables and its Receipts

<<control>>
Waiter
+seat(GameState state, Customer c): String
+takeOrder(GameState state, Customer c): String
+serve(GameState state, Customer c, int turn): String

<<entity>>
Receipt
-int customerId
-int cents
-int satisfaction
-int turn
+getCents(): int
+getSatisfaction(): int

<<control>>
Chef
+prepare(Order order) : String

ENCAPSULATION: status and completedUnits are private.
They change only through prepareOneUnit(), serve(), pay() and cancel(), which validate the transition.
Bill is locked at construction

<<entity>>
Table
-int id
-Customer customer
+isFree(): boolean
+seat(Customer c): void
+release(): void
+getCustomer(): Customer

<<entity>>
HappyHourPolicy
-boolean used
-int remainingTurns
+beginTurn(boolean activate): void
+endTurn(): void
+isActive(): boolean
+quote(Customer c, MenuItem item): int

<<entity>>
Order
-int id
-int customerId
-MenuItem item
-int billCents
-int completedUnits
-Status status
+canPrepare(): boolean
+prepareOneUnit(): void
+serve(): void
+pay(): void
+cancel(): void
-require(Status expected): void

<<control>>
GameSetup
+int MAX_TURNS
+getStageName(): String
+createMenu(): Menu
+createCustomer(int id): Customer
+hasArrival(int turn): boolean

<<entity>>
Dish
-String name
-int priceCents
-int preparationUnits
+getName(): String
+getPriceCents(): int
+getPreparationUnits(): int

<<entity>>
Menu
-List<MenuItem> regularItems
-List<MenuItem> combos
-List<MenuItem> items
+getItems(): List<MenuItem>
+get(String name) : MenuItem
+chooseFor(int customerId): MenuItem

<<enumeration>>
Order.Status
PLACED
PREPARING
READY
SERVED
PAID
CANCELLED

<<abstract>>
Customer
-int id
-int satisfaction
-Status status
-int servedTurn
-int readyToPayTurn
+getType(): String
+waitingLoss(): int
+chooseMeal (Menu menu): MenuItem
+adjustBill(int cents): int
+isWaitingForService(): boolean
+recoverSatisfaction(int points): void
+seat(): void
+order(): void
+serve(int turn): void
+finishEating(int turn): boolean
+canPay(int turn): boolean
+pay(int turn): void
+waitAtTurnEnd(): boolean

<<enumeration>>
Customer.Status
WAITING
SEATED
WAITING_FOR_FOOD
EATING
READY_TO_PAY
PAID
LEFT

<<interface>>
MenuItem
+getName(): String
+getPreparationUnits(): int
+getPriceCents():int

<<entity>>
RegularCustomer
+getType(): String
+waitingLoss(): int

<<entity>>
VIPCustomer
+getType(): String
+waitingLoss(): int
+adjustBill(int cents): int

<<entity>>
CriticCustomer
+getType(): String
+waitingLoss(): int
+chooseMeal(Menu menu): MenuItem

<<entity>>
ComboMeal
-List<MenuItem> components
-int priceCents
-int preparationUnits
+getComponents(): List<MenuItem>
+getName(): String
+getPriceCents(): int
+getPreparationUnits(): int

POLYMORPHISM: Dish and ComboMeal both realise MenuItem. Order and Chef use getPreparationUnits() without knowing which one it is.

POLYMORPHISM: Waiter calls chooseMeal(). HappyHourPolicy.quote(), reached through GameState.quoteBill(), calls adjustBill(). Customer.waitAtTurnEnd() calls waitingLoss().
All three are called on Customer. VIP and Critic differ by override, with no type checks.